

Session-Based Recommendations with Recurrent Neural Networks

📅 Date	March 29, 2016
≡ Journal/Conference	Conference ICLR
🕒 Ranking	A*
⚙️ Status	Done

Session-Based Recommendations with Recurrent Neural Networks

Abstract

Áp dụng RNN vào lĩnh vực mới - Hệ gợi ý

Hệ gợi ý thường gặp tình trạng:

- Gợi ý phải dựa trên dữ liệu phiên rất ngắn thay vì lịch sử người dùng dài.
 - Khi đó, các hướng tiếp cận bằng **phân rã ma trận**, dù thường được đánh giá cao nhưng lại cho kết quả không chính xác.
 - Vấn đề này thường được khắc phục trong thực tế bằng cách chuyển sang gợi ý mục tương đồng (item-to-item recommendations).
- Bài báo cho rằng việc mô hình hóa phiên có thể cho gợi ý chính xác hơn.
 - Họ đề xuất hướng tiếp cận dùng RNN cho gợi ý theo phiên.
 - Cách tiếp cận này có xét tới các khía cạnh thực tế của bài toán và có một vài thay đổi cơ bản so với RNN cổ điển.
 - Ví dụ như hàm ranking loss
- Kết quả thực nghiệm trên hai bộ dữ liệu cho thấy cải thiện đáng kể so với các phương pháp phổ biến đang được sử dụng.

1. Introduction

Gợi ý dựa trên phiên là một bài toán vẫn chưa thực sự được cộng đồng ML và ReSys quan tâm đúng mức.

- Nhiều hệ gợi ý thương mại điện tử (ví dụ như cửa hàng bán lẻ nhỏ) và phần lớn các trang tin tức, báo chí không theo dõi user-id của những người truy cập trang trong khoảng thời gian dài.
- Cookies và browser fingerprint: có thể cung cấp nhận diện người dùng ở một số mức độ nào đó → Nhưng không đáng tin, dấy lên lo ngại về quyền riêng tư.
- Nếu theo dõi được, trên các trang TMĐT, nhiều người cũng chỉ ghé thăm 1-2 phiên. Ngoài ra ở một số lĩnh vực nhất định, hành vi người dùng thường mang tính chất theo từng phiên → Các phiên con của người dùng thường được xử lý độc lập.
 - Do vậy hầu hết session-based recommender phát triển cho thương mại điện tử thường dựa trên các phương pháp mà không dùng tới profile người dùng (item-to-item similarity, co-occurrence, hoặc transition probabilities)

Hữu ích là vậy song các phương pháp này thường chỉ lấy click cuối cùng hoặc lựa chọn cuối cùng của người dùng mà bỏ qua thông tin từ những lần click trước đó.

Các phương pháp phổ biến nhất được dùng trong ReSys là factor models (mô hình phân rã ma trận?) và các phương pháp dựa trên láng giềng (Là các thuật toán như content-based, collaborative filtering)

- Mô hình phân rã hoạt động bằng việc phân rã ma trận user-item vốn rất thưa thành một tập vector d chiều với mỗi item và mỗi user trong dataset được biểu diễn bởi một vector → Bài toán gợi ý xem như một bài toán hoàn thiện/ tái tạo ma trận trong đó các latent vector dùng để điền các phần tử còn thiếu (VD như lấy tích vô hướng của các latent factor tương ứng giữa người dùng và item)
 - **Khó áp dụng trong gợi ý dựa trên phiên do không có hồ sơ người dùng.**
- Các phương pháp dựa trên láng giềng vốn dựa trên việc tính toán độ tương đồng giữa các item (hoặc user) lại dựa trên sự đồng xuất hiện của các mục trong các phiên. Các phương pháp này được sử dụng rộng rãi trong session-based ReSys.

Vài năm qua mảng DNN đạt thành công to lớn trong nhiều tác vụ: Ảnh và nhận diện tiếng nói. Trong mảng này, dữ liệu phi cấu trúc được xử lý qua nhiều lớp tích chập và các standard layer (?) gồm các đơn vị (thường là ReLU).

Mô hình dữ liệu tuần tự gần đây cũng thu hút nhiều chú ý với nhiều biến thể của mạng RNN, được lựa chọn cho loại dữ liệu này. Các ứng dụng của mô hình chuỗi rất đa dạng, từ machine translation đến conversation modeling và image captioning.

RNN bắt đầu được đưa vào ReSys. Trong công trình này họ cho rằng RNN có thể ứng dụng trong session-based RS với kết quả ấn tượng. Họ giải quyết **vấn đề phát sinh khi mô hình hóa dạng dữ liệu thưa** như vậy, đồng thời điều chỉnh RNN cho phù hợp với bối cảnh gợi ý bằng cách đưa ra **hàm ranking loss** phù hợp cho huấn luyện.

SS ReSys có nhiều điểm tương đồng với một số bài toán trong NLP về mặt mô hình hóa, miễn là đều xử lý dữ liệu dạng chuỗi.

Thông thường, item-set để lựa chọn có thể lên tới hàng chục nghìn, trăm nghìn item. Một thách thức khác là tập dữ liệu click-stream cũng khá lớn, nên đặt gánh nặng cho **thời gian huấn luyện và khả năng mở rộng**.

Trong phần lớn các bài toán truy hồi thông tin và hệ gợi ý, bài báo này quan tâm tới việc modeling và những top item mà user có thể thích → Họ sử dụng ranking loss để huấn luyện.

2. Related work

2.1. Session-based recommendation

Rất nhiều công trình trong mảng ReSys tập trung vào các mô hình chỉ hoạt động khi có sẵn user identifier và có thể xây dựng user profile rõ ràng.

- Với dạng này, matrix factorization và neighborhood models đã chiếm ưu thế trong các công trình nghiên cứu, đồng thời cũng được áp dụng trong các hệ thống trực tuyến.
 - item to item: Ma trận tương đồng item to item được tính trước trên dữ liệu phiên có sẵn (giả định các item thường click cùng nhau được xem như tương đồng)
 - đơn giản, hiệu quả, sử dụng rộng rãi.

→ Tuy nhiên cách này chỉ lấy last click của người dùng, bỏ qua past click.

Một hướng khác cho SSRS là Quy trình quyết định Markov (Markov Decision Processes - MDPs). Là mô hình cho các bài toán ra quyết định ngẫu nhiên mang tính tuần tự. Một MDP được định nghĩa bởi 4 phần tử $\langle S, A, Rwd, tr \rangle$ trong đó:

- S : Tập các trạng thái
- A : Tập các hành động
- Rwd : Hàm phần thưởng
- tr : Hàm chuyển trạng thái

Trong hệ gợi ý, các hành động có thể được xem tương đương với các gợi ý và các MDP đơn giản nhất là chuỗi Markov bậc 1, trong đó gợi ý tiếp theo có thể được tính đơn giản dựa trên transition probability giữa các item. Vấn đề chính khi dùng chuỗi Markov trong SSRS là không gian trạng thái nhanh chóng trở nên không quản lý được khi cố gắng đưa tất cả các chuỗi lựa chọn có thể có của người dùng.

Phiên bản mở rộng của khung phân rã tổng quát (General Factorization Framework - GFF) có khả năng dùng dữ liệu phiên để đưa ra gợi ý. Mô hình này biểu diễn một phiên bằng tổng các sự kiện trong phiên đó. Nó sử dụng 2 loại biểu diễn tiềm ẩn cho mỗi item.

- Một loại biểu diễn cho chính bản thân item đó
- Loại còn lại dùng để biểu diễn item như thành phần của phiên

Sau đó phiên được biểu diễn bằng trung bình vector đặc trưng của các item trong vai trò là một phần của phiên. **Nhược điểm là không xét đến thứ tự của các mục trong phiên**

2.2. Deep learning in recommenders

Một trong những phương pháp đầu tiên trong các nghiên cứu về mạng neural là việc sử dụng máy Boltzmann hạn chế (Restricted Boltzmann Machines - RBM) cho lọc cộng tác. Mô hình này được chứng minh là một trong những mô hình lọc cộng tác có hiệu suất tốt nhất.

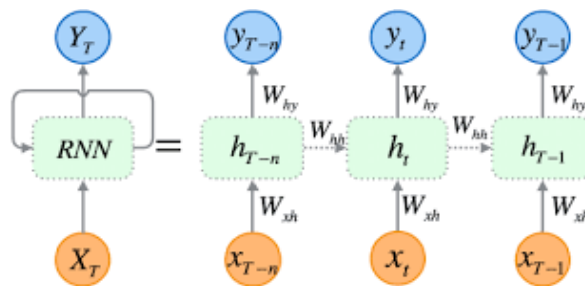
Các Deep Model đã được dùng để trích xuất từ các nội dung phi cấu trúc như nhạc/ ảnh, sau đó chúng được kết hợp với các mô hình lọc cộng tác truyền

thống. Trong nghiên cứu khác (Van et al 2013), một mạng tích chập sâu dùng để trích xuất đặc trưng từ file nhạc sau đó đưa vào factor model.

2015, Wang et al đã giới thiệu cách tiếp cận tổng quát hơn, theo đó một mạng sâu được sử dụng để trích xuất đặc trưng nội dung tổng quát từ bất kỳ loại tệp nào. Các đặc trưng này được tích hợp vào collaborative filtering model để nâng cao hiệu suất gợi ý. → Hữu ích trong bối cảnh thiếu hụt thông tin về tương tác ng dùng-item

3. Recommendations with RNNs

RNN được nghĩ ra để mô hình hóa dữ liệu tuần tự (chuỗi) có độ dài thay đổi.



Sự khác biệt chính giữa RNN và các mô hình sâu truyền thẳng thông thường là sự tồn tại của các trạng thái ẩn bên trong các đơn vị cấu thành mạng (là bộ nhớ ngắn hạn). Các RNN chuẩn cập nhật trạng thái ẩn h bằng hàm cập nhật sau:

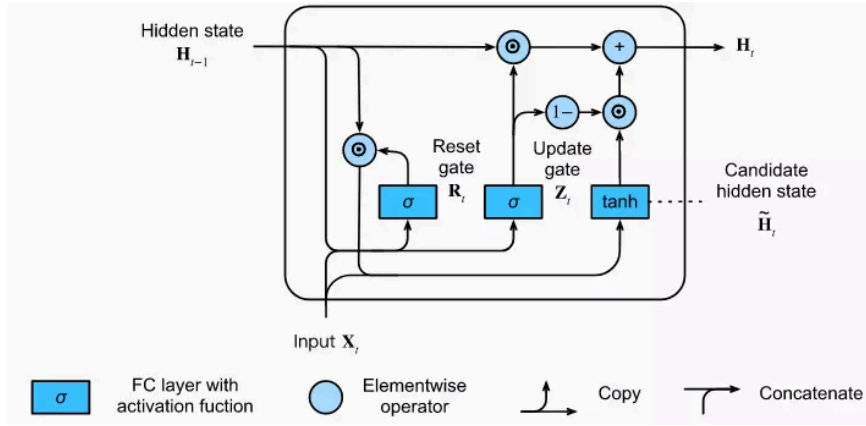
$$\mathbf{h}_t = g(W\mathbf{x}_t + U\mathbf{h}_{t-1})$$

Trong đó:

- g là hàm trơn và bị chặn (VD: sigmoid)
- x_t là đầu vào tại thời điểm t

Từ trạng thái hiện tại h_t , RNN cho ra phân phối xác suất trên phần tử tiếp theo của chuỗi.

Gated Recurrent Unit (GRU) (Mạng hồi tiếp với nút có cổng) giúp giải quyết vấn đề vanishing gradient. Các cổng trong GRU học khi nào và cập nhật trạng thái ẩn của unit bao nhiêu. Activation của GRU là phép nội suy tuyến tính giữa trạng thái kích hoạt trước đó và trạng thái kích hoạt ứng viên $\hat{h}_t$



$$\mathbf{h}_t = (1 - \mathbf{z}_t) \mathbf{h}_{t-1} + \mathbf{z}_t \hat{\mathbf{h}}_t$$

trong đó cổng cập nhật (update gate) được xác định bởi:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1})$$

trong khi hàm kích hoạt ứng viên $\hat{\mathbf{h}}_t$ được tính theo cách tương tự

$$\hat{\mathbf{h}}_t = \tanh(\mathbf{W} \mathbf{x}_t + \mathbf{U}(\mathbf{r}_t \odot \mathbf{h}_{t-1}))$$

và cuối cùng, cổng đặt lại (reset gate) $\mathbf{r}_t$ được xác định bởi:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1})$$

3.1. Customizing the GRU model

Bài báo dùng RNN dựa trên GRU trong mô hình của họ cho SSRS.

Input của mạng là trạng thái hiện tại của session, còn output là item của event tiếp theo trong session. Trạng thái của session có thể biểu diễn theo hai cách:

- Hoặc là item của event hiện tại
- Hoặc là toàn bộ các event đã xuất hiện trong session cho tới thời điểm đó.

Trong trường hợp thứ nhất, người ta sử dụng mã hóa 1 of N, tức độ dài input vector bằng số lượng item và chỉ có coordinate ứng với active item là bằng 1,

còn tất cả các coordinate khác bằng 0.

Trong trường hợp thứ hai, ta sử dụng tổng có trọng số của các biểu diễn này, trong đó các event xảy ra sớm hơn sẽ bị giảm trọng số. Để đảm bảo tính ổn định, input vector sau đó được chuẩn hóa.

Mục tiêu là tăng cường hiệu ứng ghi nhớ của các ràng buộc thứ tự cục bộ vốn không được mô hình hóa tốt bởi khả năng ghi nhớ dài hạn hơn của RNN. Báo cáo cũng thử thêm một embedding layer, nhưng cách mã hóa 1 of N luôn cho kết quả tốt hơn.

Phần cốt lõi của mạng là (các) GRU layer, và có thể thêm các feedforward layer giữa lớp cuối cùng và lớp output. Output là mức độ ưa thích dự đoán của các item, tức là khả năng mỗi item sẽ là item tiếp theo trong session. Khi sử dụng nhiều GRU layer, hidden state của lớp trước sẽ là input của lớp kế tiếp. Input cũng có thể được nối tùy chọn tới các GRU layer sâu hơn trong mạng, vì chúng tôi nhận thấy điều này cải thiện hiệu năng. Toàn bộ kiến trúc được minh họa trong Hình 1, mô tả cách biểu diễn một event đơn lẻ trong chuỗi thời gian của các event.

Vì recommender systems không phải là lĩnh vực ứng dụng chính của recurrent neural networks, chúng tôi đã điều chỉnh mạng cơ sở để phù hợp hơn với bài toán này. Chúng tôi cũng cân nhắc các khía cạnh thực tiễn để giải pháp có thể được áp dụng trong môi trường thực tế.

3.1.1 Session parallel mini batches

Các RNN trong các bài toán natural language processing thường sử dụng mini-batch theo chuỗi. Ví dụ, người ta thường dùng một sliding window trên các từ trong câu, rồi đặt các đoạn cửa sổ này cạnh nhau để tạo thành mini-batch.

Cách này không phù hợp với bài toán của chúng tôi vì:

1. độ dài của các session có thể rất khác nhau, thậm chí khác nhau nhiều hơn so với độ dài câu: có session chỉ gồm 2 event, trong khi có session có thể kéo dài tới vài trăm event;
2. mục tiêu của chúng tôi là nắm bắt cách một session tiến triển theo thời gian, nên việc cắt nhỏ thành các đoạn rời rạc là không hợp lý.

Vì vậy, chúng tôi sử dụng session-parallel mini-batches. Trước hết, chúng tôi sắp xếp thứ tự các session. Sau đó, chúng tôi lấy event đầu tiên của X session đầu tiên để tạo thành input của mini-batch đầu tiên (output mong muốn là event thứ hai của các session đang hoạt động). Mini-batch thứ hai được tạo từ các

event thứ hai, và cứ tiếp tục như vậy. Nếu một session kết thúc, session khả dụng tiếp theo sẽ được đưa vào thay thế. Các session được giả định là độc lập với nhau, vì vậy chúng tôi reset hidden state tương ứng khi việc thay thế này xảy ra. Xem Hình 2 để biết thêm chi tiết.

3.1.2 Sampling on the output

Recommender systems đặc biệt hữu ích khi số lượng item rất lớn. Ngay cả với một website bán hàng cỡ trung bình, số lượng item cũng có thể ở mức hàng chục nghìn; còn ở các hệ thống lớn hơn, việc có hàng trăm nghìn hay thậm chí vài triệu item là điều không hiếm. Nếu phải tính điểm cho mọi item ở mỗi bước, độ phức tạp của thuật toán sẽ tỷ lệ với tích của số lượng item và số lượng event, khiến nó không thể dùng được trong thực tế. Vì vậy, chúng tôi phải sample ở output và chỉ tính điểm cho một tập con nhỏ các item. Điều này cũng đồng nghĩa rằng chỉ một phần trọng số được cập nhật. Ngoài output mong muốn, chúng tôi cần tính điểm cho một số negative examples và điều chỉnh trọng số sao cho output mong muốn được xếp hạng cao.

Cách diễn giải tự nhiên đối với một event bị thiếu là user không biết đến sự tồn tại của item đó, nên không có tương tác. Tuy nhiên, cũng có một xác suất nhỏ là user thực sự biết item đó nhưng chọn không tương tác vì họ không thích nó. Item càng phổ biến thì xác suất user biết đến nó càng cao, và do đó việc thiếu tương tác càng có khả năng biểu thị sự không thích. Vì vậy, chúng ta nên sample item tỷ lệ theo độ phổ biến của chúng.

Thay vì tạo ra các mẫu riêng biệt cho từng mẫu huấn luyện, chúng tôi sử dụng các item từ những mẫu huấn luyện khác trong cùng mini-batch làm negative examples. Lợi ích của cách tiếp cận này là có thể giảm thêm thời gian tính toán bằng cách bỏ qua bước sampling tường minh. Ngoài ra, nó cũng mang lại lợi ích về mặt triển khai nhờ làm cho mã nguồn bớt phức tạp và hỗ trợ các phép toán ma trận nhanh hơn. Đồng thời, cách tiếp cận này cũng chính là một dạng popularity-based sampling, vì xác suất một item xuất hiện trong các mẫu huấn luyện khác của mini-batch tỷ lệ thuận với độ phổ biến của nó.

3.1.3 Ranking loss

Trọng tâm của recommender systems là xếp hạng item theo độ liên quan. Mặc dù bài toán cũng có thể được hiểu như một bài toán phân loại, các phương pháp learning-to-rank thường cho kết quả tốt hơn những cách tiếp cận khác. Xếp hạng có thể theo kiểu pointwise, pairwise hoặc listwise.

- Pointwise ranking ước lượng điểm số hoặc thứ hạng của từng item một cách độc lập, và hàm loss được định nghĩa sao cho thứ hạng của các item liên quan phải thấp.
- Pairwise ranking so sánh điểm số hoặc thứ hạng của từng cặp gồm một positive item và một negative item, và hàm loss buộc positive item phải có thứ hạng tốt hơn negative item.
- Listwise ranking sử dụng điểm số và thứ hạng của tất cả item rồi so sánh chúng với thứ tự lý tưởng. Vì có bao gồm bước sắp xếp, nó thường tốn kém tính toán hơn và do đó ít được sử dụng. Ngoài ra, nếu chỉ có một item liên quan — như trong trường hợp của chúng tôi — thì listwise ranking có thể được giải bằng pairwise ranking.

Chúng tôi đã đưa vào mô hình một số pointwise và pairwise ranking losses. Kết quả cho thấy pointwise ranking không ổn định với mạng này, trong khi pairwise ranking losses lại hoạt động tốt. Chúng tôi sử dụng hai hàm loss sau:

BPR

Bayesian Personalized Ranking (BPR) là một phương pháp matrix factorization sử dụng pairwise ranking loss. Nó so sánh điểm của một positive item với một negative item được sample. Trong bài này, chúng tôi so sánh điểm của positive item với nhiều item được sample và dùng trung bình của chúng làm loss. Hàm loss tại một thời điểm trong một session được định nghĩa là:

$$L_s = -\frac{1}{N_s} \sum_{j=1}^{N_s} \log(\sigma(\hat{r}_{s,i} - \hat{r}_{s,j}))$$

trong đó:

- (N_s) là số lượng mẫu âm,
- $(\hat{r}_{s,k})$ là điểm số của item (k) tại thời điểm đang xét trong session,
- (i) là item mong muốn (item tiếp theo trong session),
- (j) là các mẫu âm.

TOP1

Đây là ranking loss do chính chúng tôi đề xuất cho bài toán này. Nó là một phép xấp xỉ có regularization của relative rank của item liên quan. Relative rank của item liên quan được cho bởi:

$$\frac{1}{N_s} \sum_{j=1}^{N_s} I\{\hat{r}_{s,j} > \hat{r}_{s,i}\}$$

Chúng tôi xấp xỉ hàm chỉ thị $I\{\cdot\}$ bằng sigmoid. Tối ưu theo hàm này sẽ điều chỉnh tham số sao cho điểm của item i cao. Tuy nhiên, cách này không ổn định vì một số positive item cũng đóng vai trò là negative examples, khiến điểm số có xu hướng tăng ngày càng cao. Để tránh điều đó, chúng tôi muốn ép điểm của các negative examples nằm quanh 0. Đây là một kỳ vọng tự nhiên đối với điểm số của các negative item. Vì vậy, chúng tôi thêm một regularization term vào hàm loss. Điều quan trọng là thành phần này phải nằm trong cùng miền giá trị với relative rank và hoạt động tương tự nó. Hàm loss cuối cùng là:

$$L_s = \frac{1}{N_s} \sum_{j=1}^{N_s} \sigma(\hat{r}_{s,j} - \hat{r}_{s,i}) + \sigma(\hat{r}_{s,j}^2)$$

4. Experiments

Chúng tôi đánh giá recurrent neural network được đề xuất so với các baseline phổ biến trên hai bộ dữ liệu.

Bộ dữ liệu thứ nhất là của RecSys Challenge 2015. Bộ này chứa các click-stream của một website thương mại điện tử, đôi khi kết thúc bằng các sự kiện mua hàng. Chúng tôi sử dụng tập huấn luyện của cuộc thi và chỉ giữ lại các click events. Các session có độ dài 1 bị loại bỏ. Mạng được huấn luyện trên khoảng 6 tháng dữ liệu, bao gồm 7,966,257 session, 31,637,239 click trên 37,483 item. Chúng tôi sử dụng các session của ngày kế tiếp để kiểm thử. Mỗi session được gán hoàn toàn vào tập train hoặc test, không chia nhỏ giữa chừng. Do đặc tính của các phương pháp collaborative filtering, chúng tôi loại bỏ khỏi tập test các click vào những item không xuất hiện trong tập train. Các session có độ dài 1 cũng bị loại khỏi tập test. Sau tiền xử lý, tập test còn lại 15,324 session với 71,222 event. Bộ dữ liệu này được gọi là RSC15.

Bộ dữ liệu thứ hai được thu thập từ một nền tảng dịch vụ video OTT giống YouTube. Các sự kiện xem video trong ít nhất một khoảng thời gian nhất định được ghi nhận. Việc thu thập chỉ áp dụng cho một số khu vực và kéo dài chưa tới 2 tháng. Trong thời gian đó, hệ thống hiển thị các gợi ý item-to-item sau mỗi video ở phía bên trái màn hình. Các gợi ý này được tạo bởi nhiều thuật toán khác nhau và ảnh hưởng đến hành vi user. Các bước tiền xử lý tương tự bộ dữ liệu kia, nhưng bổ sung thêm việc loại bỏ những session quá dài vì có thể do bot tạo ra. Dữ liệu huấn luyện gồm tất cả dữ liệu ngoại trừ ngày cuối cùng của giai đoạn nói trên, với khoảng 3 triệu session, khoảng 13 triệu watch event trên 330 nghìn video. Tập test gồm các session của ngày cuối cùng, với khoảng 37

nghìn session và khoảng 180 nghìn watch event. Bộ dữ liệu này được gọi là VIDEO.

Việc đánh giá được thực hiện bằng cách đưa các event của một session vào mô hình từng cái một, rồi kiểm tra thứ hạng của item thuộc event tiếp theo. Hidden state của GRU được reset về 0 sau khi session kết thúc. Các item được sắp theo thứ tự giảm dần theo điểm số, và vị trí của chúng trong danh sách này chính là rank. Với RSC15, toàn bộ 37,483 item của tập train đều được xếp hạng. Tuy nhiên, điều này không thực tế với VIDEO do số lượng item quá lớn. Trong trường hợp đó, chúng tôi xếp hạng item mong muốn so với 30,000 item phổ biến nhất. Điều này gần như không ảnh hưởng đến đánh giá vì các item hiếm khi được truy cập thường nhận điểm thấp. Ngoài ra, việc tiền lọc dựa trên độ phổ biến cũng khá phổ biến trong các recommender systems thực tế.

Vì recommender systems chỉ có thể gợi ý một số ít item tại một thời điểm, item mà user thực sự chọn nên nằm trong một vài vị trí đầu tiên của danh sách. Do đó, thước đo đánh giá chính của chúng tôi là recall@20, tức là tỷ lệ trường hợp mà item mong muốn nằm trong top-20 item của tất cả các ca kiểm thử. Recall không xét đến thứ hạng chính xác của item miễn là nó nằm trong top-N. Điều này mô phỏng tốt một số tình huống thực tế, nơi các gợi ý không được nhấn mạnh bằng thứ tự tuyệt đối. Recall cũng thường tương quan tốt với các online KPI quan trọng như click-through rate (CTR). Thước đo thứ hai là MRR@20 (Mean Reciprocal Rank), tức là trung bình của nghịch đảo thứ hạng của các item mong muốn. Reciprocal rank được gán bằng 0 nếu rank lớn hơn 20. MRR có tính đến thứ hạng cụ thể của item, điều này quan trọng trong những trường hợp thứ tự gợi ý có ý nghĩa, ví dụ khi các item xếp hạng thấp chỉ hiện ra sau khi cuộn màn hình.

4.1 Baselines

Chúng tôi so sánh mạng được đề xuất với một tập các baseline thường được sử dụng.

- POP: Bộ dự đoán độ phổ biến, luôn gợi ý các item phổ biến nhất trong tập train. Dù rất đơn giản, nó vẫn thường là một baseline mạnh trong một số domain.
- S-POP: Baseline này gợi ý các item phổ biến nhất trong session hiện tại. Danh sách gợi ý thay đổi trong suốt session khi các item nhận thêm event. Các trường hợp hòa được phá bằng giá trị độ phổ biến toàn cục. Baseline này mạnh trong các domain có mức độ lặp lại cao.

- Item-KNN: Các item tương tự với item hiện tại được baseline này đề xuất, trong đó độ tương tự được định nghĩa là cosine similarity giữa vector session của chúng, tức là số lần đồng xuất hiện của hai item trong các session chia cho căn bậc hai của tích số lượng session mà từng item xuất hiện. Regularization cũng được thêm vào để tránh các độ tương tự cao do ngẫu nhiên ở các item hiếm. Đây là một trong những giải pháp item-to-item phổ biến nhất trong các hệ thống thực tế, cung cấp gợi ý theo kiểu *“những người đã xem item này cũng xem các item kia”*. Dù đơn giản, nó thường là một baseline mạnh.
- BPR-MF: BPR-MF là một trong những phương pháp matrix factorization thường được dùng. Nó tối ưu một pairwise ranking objective function thông qua SGD. Matrix factorization không thể được áp dụng trực tiếp cho session-based recommendations, vì các session mới không có sẵn feature vector được tính trước. Tuy nhiên, chúng ta có thể khắc phục điều này bằng cách dùng trung bình các item feature vector của các item đã xuất hiện trong session cho tới thời điểm đó làm user feature vector. Nói cách khác, chúng tôi lấy trung bình độ tương tự giữa feature vector của item được đề xuất và các item đã có trong session cho tới lúc đó.

Bảng 1 cho thấy kết quả của các baseline. Cách tiếp cận Item-KNN vượt trội rõ ràng so với các phương pháp còn lại.

4.2 Parameter and Structure optimization

Chúng tôi tối ưu các hyperparameter bằng cách chạy 100 thí nghiệm tại các điểm được chọn ngẫu nhiên trong không gian tham số cho từng bộ dữ liệu và từng hàm loss. Cấu hình tốt nhất sau đó tiếp tục được tinh chỉnh bằng cách tối ưu từng tham số riêng lẻ. Số lượng hidden units được đặt là 100 trong tất cả các trường hợp. Các tham số tốt nhất sau đó được dùng với các hidden layer có kích thước khác nhau. Quá trình tối ưu được thực hiện trên một validation set riêng. Sau đó, các mạng được huấn luyện lại trên tập train cộng validation và được đánh giá trên tập test cuối cùng.

Các cấu hình tốt nhất được tóm tắt trong Bảng 2. Các weight matrix được khởi tạo bằng các số ngẫu nhiên lấy đều trong khoảng $[-x, x]$, trong đó x phụ thuộc vào số hàng và số cột của ma trận. Chúng tôi thử nghiệm cả rmsprop và adagrad, và nhận thấy adagrad cho kết quả tốt hơn.

Chúng tôi cũng thử ngắn gọn các loại unit khác ngoài GRU. Kết quả cho thấy cả RNN unit cổ điển lẫn LSTM đều hoạt động kém hơn.

Chúng tôi thử nhiều hàm loss khác nhau. Các hàm loss dựa trên pointwise ranking, như cross-entropy và tối ưu MRR, thường không ổn định ngay cả khi có regularization. Ví dụ, với cross-entropy, trong 100 lần chạy ngẫu nhiên chỉ có 10 mạng ổn định về mặt số học trên RSC15 và 6 mạng trên VIDEO. Chúng tôi cho rằng nguyên nhân là do mô hình cố gắng độc lập nâng điểm cho các target item, trong khi lực đẩy âm đối với các negative sample lại khá nhỏ. Ngược lại, các hàm loss dựa trên pairwise ranking hoạt động tốt. Chúng tôi nhận thấy hai hàm loss được giới thiệu ở Phần 3 (BPR và TOP1) cho kết quả tốt nhất.

Nhiều kiến trúc khác nhau đã được kiểm tra và một GRU layer duy nhất cho kết quả tốt nhất. Việc thêm các layer bổ sung luôn làm hiệu năng tệ hơn xét theo cả training loss, recall và MRR trên tập test. Chúng tôi cho rằng điều này là do tuổi thọ của session nói chung khá ngắn, không đòi hỏi nhiều thang thời gian với độ phân giải khác nhau để biểu diễn phù hợp. Tuy nhiên, nguyên nhân chính xác hiện vẫn chưa rõ và cần thêm nghiên cứu.

Việc sử dụng embedding cho item cho kết quả hơi kém hơn, vì vậy chúng tôi giữ lại cách mã hóa 1-of-N. Ngoài ra, việc đưa toàn bộ các event trước đó của session vào input thay vì chỉ event ngay trước đó cũng không mang lại độ chính xác cao hơn; điều này không gây ngạc nhiên vì GRU, giống như LSTM, có cả long-term memory và short-term memory. Việc thêm các feed-forward layer sau GRU layer cũng không giúp cải thiện kết quả. Tuy nhiên, tăng kích thước của GRU layer lại cải thiện hiệu năng. Chúng tôi cũng nhận thấy việc dùng tanh làm activation function cho output layer là có lợi.

4.3 Results

Bảng 3 cho thấy kết quả của các mạng hoạt động tốt nhất. Cross-entropy trên bộ dữ liệu VIDEO với 1000 hidden units không ổn định về mặt số học, vì vậy chúng tôi không trình bày kết quả cho trường hợp đó. Các kết quả được so sánh với baseline tốt nhất là Item-KNN. Chúng tôi hiển thị kết quả với 100 và 1000 hidden units.

Thời gian chạy phụ thuộc vào tham số và bộ dữ liệu. Nói chung, chênh lệch thời gian chạy giữa phiên bản nhỏ và lớn không quá cao trên GPU GeForce GTX Titan X, và việc huấn luyện mạng có thể hoàn thành trong vài giờ. Trên CPU, mạng nhỏ hơn có thể được huấn luyện trong khoảng thời gian chấp nhận được về mặt thực tiễn. Việc huấn luyện lại thường xuyên là điều mong muốn trong recommender systems, vì user mới và item mới xuất hiện liên tục.

Cách tiếp cận dựa trên GRU cho thấy sự cải thiện đáng kể so với Item-KNN ở cả hai chỉ số đánh giá trên cả hai bộ dữ liệu, ngay cả khi số lượng unit chỉ là 100. Việc tăng số lượng unit còn tiếp tục cải thiện kết quả đối với các hàm loss pairwise, nhưng độ chính xác lại giảm đối với cross-entropy. Mặc dù cross-entropy cho kết quả tốt hơn khi dùng 100 hidden units, các biến thể dùng pairwise loss vượt qua mức này khi số unit tăng lên. Dù việc tăng số unit làm tăng thời gian huấn luyện, chúng tôi thấy rằng việc tăng từ 100 lên 1000 unit trên GPU không quá tốn kém.

Ngoài ra, hàm loss dựa trên cross-entropy được phát hiện là không ổn định về mặt số học, do mạng cố gắng riêng lẻ tăng điểm cho các target item, trong khi lực đẩy âm đối với các item khác là tương đối nhỏ. Vì vậy, chúng tôi đề xuất sử dụng một trong hai pairwise loss. TOP1 loss hoạt động nhỉnh hơn một chút trên hai bộ dữ liệu này, mang lại mức tăng độ chính xác khoảng 20–30% so với baseline tốt nhất.

5. Conclusion and Future works

Trong bài báo này, chúng tôi áp dụng một loại recurrent neural network hiện đại (GRU) vào một lĩnh vực ứng dụng mới: recommender systems. Chúng tôi chọn bài toán session-based recommendations vì đây là một lĩnh vực quan trọng về mặt thực tiễn nhưng chưa được nghiên cứu đầy đủ. Chúng tôi đã điều chỉnh GRU cơ bản để phù hợp hơn với bài toán bằng cách đưa vào session-parallel mini-batches, output sampling dựa trên mini-batch và ranking loss function. Chúng tôi cho thấy rằng phương pháp của mình có thể vượt trội đáng kể so với các baseline phổ biến được sử dụng cho bài toán này. Chúng tôi tin rằng công trình này có thể trở thành nền tảng cho cả các ứng dụng deep learning trong recommender systems lẫn session-based recommendations nói chung.

Trong tương lai gần, chúng tôi sẽ tập trung vào việc khảo sát sâu hơn mạng được đề xuất. Chúng tôi cũng có kế hoạch huấn luyện mạng trên các item representation được trích xuất tự động từ chính content của item (ví dụ: thumbnail, video, text) thay vì dùng input hiện tại.

Note

Matrix Factorization

Là cách cố gắng phân rã ma trận utility ban đầu thành 2 ma trận nhỏ hơn.

- Một ma trận biểu diễn người dùng
- Một ma trận biểu diễn item.

Mỗi người dùng và mỗi item được gán một vector đặc trưng ẩn. Khi đó mức độ yêu thích được tính bằng cách lấy độ khớp giữa vector user và vector item, thường là tích vô hướng.

Dù bằng thừa nhưng mô hình vẫn dự đoán được các ô còn thiếu khá tốt. Nhưng cách này lại phụ thuộc quá nhiều vào profile người dùng, không phù hợp với SSRS

Markov Decision Process - MDP

Là mô hình cho bài toán ra quyết định tuần tự. Một MDP gồm tập trạng thái (S), tập hành động (A), phần thưởng (Rwd) và hàm chuyển trạng thái (tr)

Trong RS:

- state: Tình huống hiện tại của user (Ví dụ user vừa xem áo sơ mi rồi click quần jean)
- action: Gợi ý một item nào đó
- reward: User có click hay mua không
- transition: Sau khi gợi ý xong, trạng thái user thay đổi như thế nào?

Giống mô hình chuỗi markov

General Factorization Framework

GFF là khung tổng quát cho các mô hình factorization. Nó cố gắng mở rộng matrix factorization để xử lý thêm nhiều loại thông tin như user, item, session, context.

Phương pháp này biểu diễn một phiên bằng cách cộng hoặc trung bình các item đã xuất hiện trong session.

Ví dụ một session có các item A B C thì session được biểu diễn: $\text{vector_session} = \text{mean}(\text{vec_A}, \text{vec_B}, \text{vec_C})$

Ở bài báo này, mỗi item có 2 vector:

- Vector biểu diễn bản thân item
- Vector biểu diễn item khi nó là một phần của session

Điều này giúp mô hình học rằng: item đứng riêng là một chuyện nhưng khi xuất hiện trong ngữ cảnh ss lại là chuyện khác

Ưu điểm là tận dụng được dữ liệu session và không nhất thiết cần lưu hồ sơ user lâu dài. Nhưng nhược điểm là không quan tâm thứ tự.